ShopKart — E-Commerce Platform Documentation

oday

Contents

ShopKart — E-Commerce Platform **3**
- Table of Contents . . . 3
- Key Features . . . 3
- Tech Stack . . . 3
- Repository Layout . . . 3
- Requirements . . . 4
- Installation & Setup . . . 4
  - 1. Backend . . . 4
  - 2. Frontend . . . 4
- Quick Start . . . 5
- Configuration . . . 5
- Documentation . . . 5
- License . . . 5

API Reference **6**
- Table of Contents . . . 6
- Conventions . . . 6
- Authentication (`/api/auth`) . . . 6
  - `POST /api/auth/signup` . . . 6
  - `POST /api/auth/signin` . . . 7
- User (`/api/user`) . . . 7
- Addresses (`/api/addresses`) . . . 8
- Categories (`/api/categories`) . . . 8
- Products (`/api/products`) . . . 8
- Product Images (`/api/products/{productId}/images`) . . . 9
- Cart (`/api/cart`) . . . 10
- Wishlist (`/api/wishlist`) . . . 10
- Coupons (`/api/coupons`) . . . 10
- Orders (`/api/orders`) . . . 11
- Payments (`/api/payments`) . . . 12
- Reviews (`/api/reviews`) . . . 12
- Errors . . . 12

Usage Guide **14**
- Table of Contents . . . 14
- Authentication Flow . . . 14
- Browsing & Searching Products . . . 14
- Cart & Checkout Flow . . . 15
- Managing Products as a Seller . . . 15
- Order Fulfillment (Admin/Seller) . . . 16

Reviews & Wishlist 16
Error Handling Patterns 17
Configuration Options 17

Architecture & Reference **18**
Table of Contents 18
System Overview 18
Backend Architecture 18
Authentication & Authorization 19
Frontend Architecture 19
Data Model Overview 19
Known Limitations 19
Troubleshooting / FAQ 20

ShopKart — E-Commerce Platform

A full-stack e-commerce application with a Spring Boot REST API backend and a React SPA frontend. Supports customer shopping (browse, cart, wishlist, checkout, orders, reviews) and seller/admin management (products, categories, coupons, orders, users).

Table of Contents

- Key Features
- Tech Stack
- Repository Layout
- Requirements
- Installation & Setup
- Quick Start
- Configuration
- Documentation
- License

Key Features

Customer - Signup/login with JWT authentication - Browse, search, and filter products by category - Shopping cart and wishlist - Address book management - Checkout with coupon codes and wallet balance - Order history, tracking, and cancellation - Product reviews and ratings

Seller / Admin - Product, variant, and product-image management (with reordering and primary image) - Category management - Coupon creation and management - Order fulfillment (status updates) - User and role management, seller/customer profile views (admin only)

Tech Stack

Layer	Technology
Backend	Java 17, Spring Boot 3.3.1 (Web, Data JPA, Security, Validation)
Database	MySQL
Auth	JWT (`jjwt` 0.11.5)
Mapping	MapStruct + Lombok
API Docs	springdoc-openapi (Swagger UI)
Frontend	React 19, TypeScript, Vite 8
Routing/Data	TanStack Router, TanStack Query
Styling	Tailwind CSS v4
HTTP Client	Axios

Repository Layout

This project spans two repositories:

```
E_COMMERCE_BACKEND/          Spring Boot REST API
├── src/main/java/com/ecommerce/
│   ├── controllers/         REST endpoints
│   ├── services/            Business logic (interfaces + impl/)
│   ├── repositories/        Spring Data JPA repositories
```

```
│   ├── models/              JPA entities
│   ├── dto/                 Request/response DTOs
│   ├── mapper/               MapStruct entity <-> DTO mappers
│   ├── security/             JWT filter, security config, user details
│   ├── config/               Swagger, MVC, data seeding config
│   └── exception/            Global exception handling
└── src/main/resources/application.properties

E_COMMERCE_FRONTEND/          React SPA
├── src/routes/               File-based routes (TanStack Router)
├── src/components/           Shared UI components
├── src/lib/api/               Axios client + endpoint functions
├── src/lib/auth-context.tsx  Auth state (JWT, current user)
└── vite.config.ts            Dev server + proxy config
```

Requirements

- **Backend:** Java 17+, Maven, MySQL 8+
- **Frontend:** Node.js 18+, npm

Installation & Setup

1. Backend

```
git clone https://github.com/2310Eswar/E_COMMERCE_BACKEND.git
cd E_COMMERCE_BACKEND

# Create the database
mysql -u root -p -e "CREATE DATABASE e_commerce;"
```

Set your local DB credentials and JWT secret as environment variables (recommended, rather than relying on the checked-in defaults — see Configuration):

```
export DB_URL="jdbc:mysql://localhost:3306/e_commerce?useSSL=false&serverTimezone=UTC&allowPubl
export DB_USERNAME=root
export DB_PASSWORD=your_password
export JWT_SECRET=$(openssl rand -hex 32)
```

Build and run:

```
./mvnw spring-boot:run
```

The API starts on **http://localhost:8081**. Hibernate auto-creates/updates tables (`ddl-auto=update`).

2. Frontend

```
git clone https://github.com/2310Eswar/E_COMMERCE_FRONTEND.git
cd E_COMMERCE_FRONTEND
npm install
```

Optionally set the API base URL (defaults to `http://localhost:8081`):

```
echo "VITE_API_BASE_URL=http://localhost:8081" > .env
```

Run the dev server:

```
npm run dev
```

The app starts on **http://localhost:5173** and proxies `/api` and `/images` requests to the backend during development (see `vite.config.ts`).

Quick Start

1. Start MySQL, then the backend (`./mvnw spring-boot:run`).
2. Start the frontend (`npm run dev`).
3. Visit `http://localhost:5173`, sign up for a customer account, and start browsing products.
4. To act as a seller/admin, update a user's role via `PUT /api/user/admin/{userId}/role` (requires an existing ADMIN account — see `DataSeeder` for any seeded default users).
5. Explore the raw API directly via Swagger UI at `http://localhost:8081/swagger-ui.html`.

Configuration

Backend configuration lives in `src/main/resources/application.properties`. All values are overridable via environment variables:

Property	Env Var	Default
Server port	—	`8081`
DB URL	`DB_URL`	`jdbc:mysql://localhost:3306/e_commerce.`
DB username	`DB_USERNAME`	`root`
DB password	`DB_PASSWORD`	*(set your own — do not use the checked-in default)*
JWT secret	`JWT_SECRET`	*(set your own — do not use the checked-in default)*
JWT expiry	—	`86400000` ms (24h)
Upload directory	`UPLOAD_DIR`	`uploads/`
Max upload size	—	`5MB`

> **Security note:** `application.properties` currently checks in default values for `DB_PASSWORD` and `JWT_SECRET`. Before deploying anywhere beyond local development, override both via environment variables and remove the hardcoded fallbacks (or at minimum, rotate them and add the file to `.gitignore` for any environment-specific overrides).

Documentation

- `docs/API_REFERENCE.md` — full endpoint reference
- `docs/USAGE_GUIDE.md` — common workflows and code examples
- `docs/ARCHITECTURE.md` — architecture, auth flow, limitations, FAQ

License

No license file is currently present in either repository. Add a `LICENSE` file to clarify usage terms before sharing or open-sourcing this project. -e

API Reference

Base URL: `http://localhost:8081` (configurable via `server.port`)

Interactive docs: `GET /swagger-ui.html` · OpenAPI spec: `GET /api-docs`

Table of Contents

- Conventions
- Authentication
- User
- Addresses
- Categories
- Products
- Product Images
- Cart
- Wishlist
- Coupons
- Orders
- Payments
- Reviews
- Errors

Conventions

- All request/response bodies are JSON unless noted (image uploads use `multipart/form-data`).
- Protected endpoints require an `Authorization: Bearer <token>` header, obtained from `/api/auth/signin` or `/api/auth/signup`.
- Roles are `ROLE_CUSTOMER`, `ROLE_SELLER`, `ROLE_ADMIN`. Endpoints marked **Auth: none** are public; others list the roles allowed.
- Paginated endpoints accept standard Spring `Pageable` query params: `?page=0&size=20&sort=field,asc`
- IDs (`{id}`, `{productId}`, etc.) are numeric (`Long`).

Authentication (`/api/auth`)

`POST /api/auth/signup`

Auth: none

Registers a new customer account.

Request body:

```
{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "Passw0rd!",
  "dateOfBirth": "1995-06-01",
  "mobileNumber": "9876543210",
  "gender": "FEMALE"
}
```

Validation: `email` must be valid; `password` must be 8+ chars with upper, lower, digit, and special character; `dateOfBirth` must be in the past; `mobileNumber` must be exactly 10 digits.

Response `200`: same shape as the signin response below.

`POST /api/auth/signin`

Auth: none

Request body:

```
{ "email": "jane@example.com", "password": "Passw0rd!" }
```

Response `200`:

```
{
  "token": "<jwt>",
  "id": 1,
  "email": "jane@example.com",
  "name": "Jane Doe",
  "roles": ["ROLE_CUSTOMER"]
}
```

User (`/api/user`)

Method	Path	Auth	Description
GET	`/profile`	any authenticated user	Get current user's profile
PUT	`/profile`	any authenticated user	Update current user's profile
POST	`/addresses`	any authenticated user	Add an address to the current user
GET	`/addresses`	any authenticated user	List current user's addresses
GET	`/admin/all`	ADMIN	List all users
PUT	`/admin/{userId}/role?role=`	ADMIN	Change a user's role
GET	`/admin/sellers/{sellerId}/profile`	ADMIN	Seller profile detail
GET	`/admin/customers/{customerId}/profile`	ADMIN	Customer profile detail
DELETE	`/admin/{userId}`	ADMIN	Delete a user
PUT	`/admin/{userId}/status`	ADMIN	Toggle active/inactive status
POST	`/wallet/add?amount=`	CUSTOMER	Add funds to wallet balance

Example — update profile:

```
PUT /api/user/profile
Authorization: Bearer <jwt>
Content-Type: application/json

{ "name": "Jane A. Doe", "mobileNumber": "9876500000" }
```

Addresses (`/api/addresses`)

Method	Path	Auth	Description
POST	`/`	any authenticated user	Add address
PUT	`/{id}`	any authenticated user	Update address
DELETE	`/{id}`	any authenticated user	Delete address
GET	`/{id}`	any authenticated user	Get address by ID
GET	`/`	any authenticated user	List current user's addresses

Request body (`AddressRequest`):

```
{
  "line1": "221B Baker Street",
  "city": "Chennai",
  "state": "Tamil Nadu",
  "postalCode": "600001",
  "country": "India",
  "isDefault": true
}
```

(Exact fields depend on `AddressRequest`; check the DTO for the authoritative list.)

Categories (`/api/categories`)

Method	Path	Auth	Description
POST	`/`	ADMIN, SELLER	Create category
PUT	`/{id}`	ADMIN, SELLER	Update category
DELETE	`/{id}`	ADMIN, SELLER	Delete category
GET	`/{id}`	none	Get category by ID
GET	`/`	none	List all categories
GET	`/admin/all`	ADMIN, SELLER	List categories owned by the current seller (or all, for admin)

Products (`/api/products`)

Method	Path	Auth	Description
GET	`/`	none	Paginated list of active products
GET	`/search?keyword=`	none	Paginated keyword search

Method	Path	Auth	Description
GET	`/category/{categoryId}`	none	Paginated products in a category
GET	`/deals`	none	Products currently on deal/discount
GET	`/{id}`	none	Product detail
POST	`/`	ADMIN, SELLER	Create product
POST	`/{id}/main-image`	ADMIN, SELLER	Upload main image (`multipart/form-data`, field `file`)
PUT	`/{id}`	ADMIN, SELLER	Update product
PUT	`/{id}/variants/{variantId}/pricing`	ADMIN, SELLER	Update a variant's pricing
DELETE	`/{id}`	ADMIN, SELLER	Delete product
GET	`/admin/all`	ADMIN, SELLER	Paginated list of the seller's own products (all, for admin)

Request body (`ProductRequest`):

```
{
  "name": "Wireless Mouse",
  "description": "Ergonomic wireless mouse with USB receiver",
  "basePrice": 799.00,
  "imageUrl": null,
  "categoryId": 3,
  "variants": [
    { "sku": "WM-BLK", "attributes": "Color: Black", "stock": 100 }
  ]
}
```

(`variants` shape depends on `ProductVariantRequest` — confirm field names against that DTO.)

Update variant pricing (`PricingRequest`):

```
PUT /api/products/12/variants/45/pricing
{ "price": 749.00, "discountPercent": 10 }
```

Product Images (`/api/products/{productId}/images`)

All endpoints require **ADMIN or SELLER**, except `GET` which is public (`permitAll`).

Method	Path	Description
POST	`/`	Upload an image (`multipart/form-data`, field `file`)
GET	`/`	List images for a product
DELETE	`/{imageId}`	Delete an image
PATCH	`/{imageId}/primary`	Set an image as the primary/thumbnail image

Method	Path	Description
PATCH	`/reorder`	Reorder images — body is a JSON array of image IDs, e.g. `[5, 3, 8]`

Example — upload:

```
curl -X POST http://localhost:8081/api/products/12/images \
  -H "Authorization: Bearer <jwt>" \
  -F "file=@/path/to/photo.jpg"
```

Cart (`/api/cart`)

Auth: CUSTOMER for all endpoints.

Method	Path	Description
GET	`/`	Get current user's cart
POST	`/add`	Add an item to the cart
DELETE	`/items/{variantId}`	Remove an item by product variant ID
PUT	`/items/{variantId}`	Update quantity of an item
DELETE	`/clear`	Empty the cart

Add to cart (`CartRequest`):

```
{ "variantId": 45, "quantity": 2 }
```

Update quantity: same body shape; `quantity` is read from the body, `variantId` is redundant with the path but present for consistency.

Wishlist (`/api/wishlist`)

Auth: CUSTOMER for all endpoints.

Method	Path	Description
GET	`/`	List product IDs in the wishlist
POST	`/{productId}`	Add a product
DELETE	`/{productId}`	Remove a product
DELETE	`/clear`	Clear the wishlist

Coupons (`/api/coupons`)

Method	Path	Auth	Description
POST	`/`	ADMIN, SELLER	Create coupon

Method	Path	Auth	Description
PUT	`/{id}`	ADMIN, SELLER	Update coupon
DELETE	`/{id}`	ADMIN, SELLER	Deactivate coupon (soft delete)
GET	`/`	none	List all coupons
GET	`/admin/all`	ADMIN, SELLER	List coupons owned by the current seller
GET	`/active`	none	List currently active coupons

Request body (`CouponRequest`):

```
{
  "code": "WELCOME10",
  "discountPercent": 10,
  "expiryDate": "2026-12-31",
  "minOrderValue": 500.00
}
```

(Confirm exact fields against `CouponRequest`.)

Orders (`/api/orders`)

Method	Path	Auth	Description
POST	`/place/{addressId}?couponCode=`	CUSTOMER	Place an order using the current cart, a saved address, and an optional coupon code
GET	`/{orderId}`	any authenticated user	Get order detail
GET	`/my-orders`	any authenticated user	List the current user's orders
GET	`/`	ADMIN, SELLER	Paginated list of all orders
GET	`/admin/all`	ADMIN, SELLER	Paginated list of orders for the current seller (or all, for admin)
PUT	`/{orderId}/status?status=`	ADMIN, SELLER	Update order status
PUT	`/{orderId}/cancel`	any authenticated user	Cancel an order

Order status values: `PLACED`, `CONFIRMED`, `PENDING`, `PACKED`, `SHIPPED`, `OUT_FOR_DELIVERY`, `DELIVERED`, `CANCELLED`, `RETURNED`

Example — place order:

```
POST /api/orders/place/7?couponCode=WELCOME10
Authorization: Bearer <jwt>
```

Example — update status:

```
PUT /api/orders/101/status?status=SHIPPED
Authorization: Bearer <jwt>
```

Payments (`/api/payments`)

Method	Path	Auth	Description
POST	`/`	any authenticated user	Record/process a payment
GET	`/{id}`	any authenticated user	Get payment by ID
GET	`/order/{orderId}`	any authenticated user	Get payment by order ID

Payment status values: `PENDING`, `COMPLETED`, `FAILED`, `REFUNDED`

Request body (`PaymentRequest`):

```
{ "orderId": 101, "method": "CARD", "amount": 1499.00 }
```

(Confirm exact fields against `PaymentRequest`.)

Reviews (`/api/reviews`)

Method	Path	Auth	Description
POST	`/`	any authenticated user	Add a review
PUT	`/{id}`	any authenticated user	Update a review
DELETE	`/{id}`	any authenticated user	Delete a review
GET	`/product/{productId}`	none	List reviews for a product
GET	`/user/{userId}`	none	List reviews by a user

Request body (`ReviewRequest`):

```
{ "productId": 12, "rating": 5, "comment": "Great product, fast delivery." }
```

(Confirm exact fields against `ReviewRequest`.)

Errors

All errors return a JSON body via the global exception handler:

```
{
  "error": "NOT_FOUND",
  "message": "Product not found with id: 999",
  "details": null
}
```

HTTP Status	`error` code	When
404	`NOT_FOUND`	Resource doesn't exist (`ResourceNotFoundException`)
403	`FORBIDDEN`	Authenticated but not authorized for the action
400	`VALIDATION_ERROR`	`@Valid` request body failed validation — `details` contains a field → message map
400	`BAD_REQUEST`	Type mismatch on a path/query param, or an illegal argument
500	`INTERNAL_ERROR`	Unhandled exception

Validation error example:

```
{
  "error": "VALIDATION_ERROR",
  "message": "Validation failed",
  "details": {
    "email": "must be a well-formed email address",
    "password": "Password must be at least 8 characters long, contain at least one digit, one l
  }
}
```

-e

Usage Guide

Practical, end-to-end examples for the most common flows in ShopKart. Examples use `curl` for the raw API and TypeScript/Axios for frontend integration.

Table of Contents

- Authentication Flow
- Browsing & Searching Products
- Cart & Checkout Flow
- Managing Products as a Seller
- Order Fulfillment (Admin/Seller)
- Reviews & Wishlist
- Error Handling Patterns
- Configuration Options

Authentication Flow

1. Sign up:

```
curl -X POST http://localhost:8081/api/auth/signup \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Jane Doe",
    "email": "jane@example.com",
    "password": "Passw0rd!",
    "mobileNumber": "9876543210"
  }'
```

2. Sign in and capture the token:

```
TOKEN=$(curl -s -X POST http://localhost:8081/api/auth/signin \
  -H "Content-Type: application/json" \
  -d '{"email":"jane@example.com","password":"Passw0rd!"}' | jq -r '.token')
```

3. Use the token on subsequent requests:

```
curl http://localhost:8081/api/user/profile \
  -H "Authorization: Bearer $TOKEN"
```

In the frontend, `src/lib/auth-context.tsx` wraps this: `login(jwt)` persists the token and user to `localStorage` (`shopkart_token` / `shopkart_user`), and the Axios instance in `src/lib/api/client.ts` automatically attaches `Authorization: Bearer <token>` to every request. On a `401` response, it clears the stored session — pair this with a redirect to `/login` in your route guards (see `RequireAuth.tsx`).

Browsing & Searching Products

Products are paginated using Spring's `Pageable`, so pass `page`, `size`, and optionally `sort` as query params:

```
# First page, 12 per page, newest first
curl "http://localhost:8081/api/products?page=0&size=12&sort=id,desc"

# Keyword search
curl "http://localhost:8081/api/products/search?keyword=mouse&page=0&size=12"
```

```
# Products in a category
curl "http://localhost:8081/api/products/category/3?page=0&size=12"

# Current deals (not paginated)
curl "http://localhost:8081/api/products/deals"
```

Cart & Checkout Flow

1. Add an item to the cart (requires CUSTOMER role):

```
curl -X POST http://localhost:8081/api/cart/add \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"variantId": 45, "quantity": 2}'
```

2. Review the cart:

```
curl http://localhost:8081/api/cart -H "Authorization: Bearer $TOKEN"
```

3. Add a shipping address (if not already saved):

```
curl -X POST http://localhost:8081/api/addresses \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"line1":"221B Baker Street","city":"Chennai","state":"Tamil Nadu","postalCode":"600001",
```

4. Place the order against a saved address, with an optional coupon:

```
curl -X POST "http://localhost:8081/api/orders/place/7?couponCode=WELCOME10" \
  -H "Authorization: Bearer $TOKEN"
```

5. Pay for the order:

```
curl -X POST http://localhost:8081/api/payments \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"orderId": 101, "method": "CARD", "amount": 1499.00}'
```

6. Track it:

```
curl http://localhost:8081/api/orders/101 -H "Authorization: Bearer $TOKEN"
```

Note the cart is tied to the authenticated user server-side — there's no separate "cart ID" to manage on the client.

Managing Products as a Seller

Requires a `ROLE_SELLER` or `ROLE_ADMIN` account.

1. Create a category (or reuse an existing one via `GET /api/categories`):

```
curl -X POST http://localhost:8081/api/categories \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name": "Electronics"}'
```

2. Create a product:

```
curl -X POST http://localhost:8081/api/products \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{
    "name": "Wireless Mouse",
```

```
    "description": "Ergonomic wireless mouse",
    "basePrice": 799.00,
    "categoryId": 3,
    "variants": []
  }'
```

3. Upload a main image:

```
curl -X POST http://localhost:8081/api/products/12/main-image \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@/path/to/mouse.jpg"
```

4. Upload additional gallery images and set one as primary:

```
curl -X POST http://localhost:8081/api/products/12/images \
  -H "Authorization: Bearer $TOKEN" -F "file=@/path/to/angle2.jpg"

curl -X PATCH http://localhost:8081/api/products/12/images/8/primary \
  -H "Authorization: Bearer $TOKEN"
```

5. Update variant pricing (e.g. run a discount):

```
curl -X PUT http://localhost:8081/api/products/12/variants/45/pricing \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"price": 749.00, "discountPercent": 10}'
```

Order Fulfillment (Admin/Seller)

```
# List orders for your store, paginated
curl "http://localhost:8081/api/orders/admin/all?page=0&size=20" \
  -H "Authorization: Bearer $TOKEN"

# Advance an order through its lifecycle
curl -X PUT "http://localhost:8081/api/orders/101/status?status=PACKED" \
  -H "Authorization: Bearer $TOKEN"
curl -X PUT "http://localhost:8081/api/orders/101/status?status=SHIPPED" \
  -H "Authorization: Bearer $TOKEN"
curl -X PUT "http://localhost:8081/api/orders/101/status?status=DELIVERED" \
  -H "Authorization: Bearer $TOKEN"
```

Valid transitions aren't enforced client-side by this guide — check `OrderServiceImpl` for any server-side rules on which status transitions are permitted.

Reviews & Wishlist

```
# Leave a review
curl -X POST http://localhost:8081/api/reviews \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"productId": 12, "rating": 5, "comment": "Works great."}'

# Add/remove wishlist items
curl -X POST http://localhost:8081/api/wishlist/12 -H "Authorization: Bearer $TOKEN"
curl -X DELETE http://localhost:8081/api/wishlist/12 -H "Authorization: Bearer $TOKEN"
```

Error Handling Patterns

The frontend centralizes error parsing in `apiErrorMessage()` (`src/lib/api/client.ts`), which extracts a human-readable message from any `ErrorResponse` the backend returns:

```
import { api, apiErrorMessage } from "@/lib/api/client";

try {
  await api.post("/api/cart/add", { variantId: 45, quantity: 2 });
} catch (err) {
  toast.error(apiErrorMessage(err, "Could not add item to cart"));
}
```

For validation errors (`VALIDATION_ERROR`), the `details` field is a map of `fieldName -> message` — useful for highlighting individual form fields:

```
catch (err) {
  const e = err as AxiosError<{ details?: Record<string,string> }>;
  const fieldErrors = e.response?.data?.details ?? {};
  // e.g. setFormErrors(fieldErrors)
}
```

A `401` response is handled globally by the Axios interceptor (clears the stored session) — wrap authenticated routes in `RequireAuth` so the user is redirected to `/login` rather than seeing a broken page.

Configuration Options

Setting	Where	Notes
API base URL (frontend)	`VITE_API_BASE_URL` env var	Defaults to `http://localhost:8081`
DB connection (backend)	`DB_URL`, `DB_USERNAME`, `DB_PASSWORD` env vars	See `application.properties`
JWT secret & expiry	`JWT_SECRET` env var / `app.jwt.expirationMs`	Expiry defaults to 24h
Upload directory	`UPLOAD_DIR` env var	Defaults to `uploads/` relative to working dir
Max upload size	`spring.servlet.multipart.max-file-size`	5MB

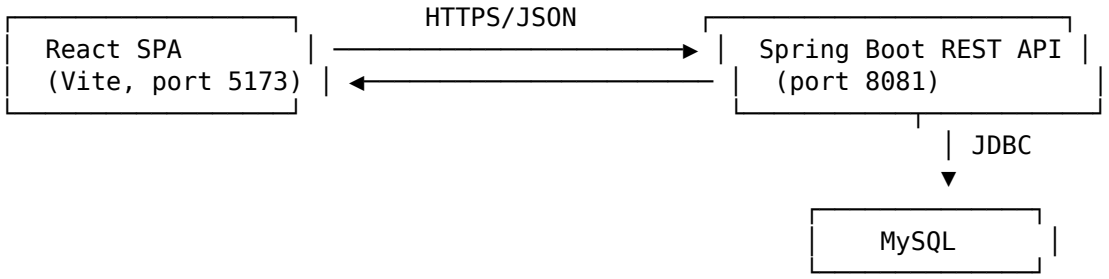
-e

Architecture & Reference

Table of Contents

- System Overview
- Backend Architecture
- Authentication & Authorization
- Frontend Architecture
- Data Model Overview
- Known Limitations
- Troubleshooting / FAQ

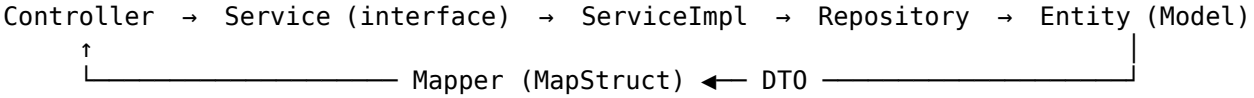
System Overview



In development, Vite proxies `/api/*` and `/images/*` requests to the backend (see `vite.config.ts`), so the frontend and backend can be run on separate ports without CORS configuration. In production, you would typically either serve the built frontend from the same origin behind a reverse proxy, or configure CORS explicitly on the backend.

Backend Architecture

The backend follows a standard layered Spring Boot structure:



- **Controllers** (`controllers/`) — thin REST endpoints; handle request binding, `@Valid` validation, and `@PreAuthorize` role checks. They call into service interfaces and never touch repositories directly.
- **Services** (`services/`, `services/impl/`) — business logic (pricing, order placement, cart merging, coupon validation, etc.), defined as interfaces with `*Impl` implementations for testability.
- **Repositories** (`repositories/`) — Spring Data JPA interfaces over MySQL.
- **Models** (`models/`) — JPA entities (`User`, `Product`, `Order`, `Cart`, etc.).
- **DTOs** (`dto/`, `dto/request/`, `dto/response/`) — request/response shapes decoupled from entities, so internal model changes don't automatically change the API contract.
- **Mappers** (`mapper/`) — MapStruct-generated entity ↔ DTO conversions.
- **Security** (`security/`) — `JwtAuthFilter`, `JwtUtils`, `SecurityConfig`, `UserDetailsImpl`/`UserDetailsServiceImpl`.
- **Config** (`config/`) — `SwaggerConfig`, `WebMvcConfig` (e.g. static file serving for uploads), and `DataSeeder` (seeds initial data on startup, e.g. default admin/categories — check this class to see what's pre-populated).
- **Exception handling** (`exception/`) — a single `@RestControllerAdvice` (`GlobalExceptionHandler`) maps known exceptions to consistent error JSON (see API Reference → Errors).

Authentication & Authorization

1. `POST /api/auth/signup` or `/signin` returns a signed JWT (`JwtResponse`), including the user's roles.
2. The client stores the token and sends it as `Authorization: Bearer <token>` on every subsequent request.
3. `JwtAuthFilter` intercepts each request, validates the token via `JwtUtils`, and loads a `UserDetailsImpl` into the Spring Security context.
4. Controller methods enforce authorization with `@PreAuthorize("hasRole('...')")` / `hasAnyRole(...)`, either at the class level (e.g. `CartController`, `WishlistController` — CUSTOMER-only) or per-method (e.g. product/category writes restricted to ADMIN/SELLER).
5. Endpoints with no `@PreAuthorize` annotation are reachable by any authenticated (or in some cases, any) request — check each controller's class- and method-level annotations for the authoritative rule, since defaults depend on `SecurityConfig`.

Roles: `ROLE_CUSTOMER`, `ROLE_SELLER`, `ROLE_ADMIN` (see `enums/Role.java`). Role changes are performed via `PUT /api/user/admin/{userId}/role` (ADMIN only).

Frontend Architecture

- **Routing:** File-based routes under `src/routes/` via TanStack Router (`routeTree.gen.ts` is generated — don't hand-edit it). Dot-separated filenames map to nested paths and dynamic segments, e.g. `admin.products.$id.edit.tsx` → `/admin/products/:id/edit`.
- **Data fetching:** TanStack Query wraps the Axios calls in `src/lib/api/endpoints.ts`, giving caching, refetching, and loading/error states out of the box.
- **Auth state:** `src/lib/auth-context.tsx` provides a `useAuth()` hook (`user`, `token`, `isAuthed`, `login`, `logout`), backed by `localStorage`.
- **Route protection:** `RequireAuth.tsx` gates authenticated-only routes (and presumably role-specific admin routes — check its implementation for how it distinguishes customer vs. admin/seller access).
- **Styling:** Tailwind CSS v4 via the `@tailwindcss/vite` plugin — utility classes directly in components, no separate CSS-in-JS layer.
- **Cart/wishlist utilities:** `src/lib/cart-utils.ts` and `src/lib/wishlist.ts` hold client-side helper logic around those features.

Data Model Overview

Core entities (see `models/`): `User` (with `CustomerDetails` / `SellerDetails`), `Address`, `Category`, `Product`, `ProductVariant`, `ProductImage`, `Cart`/`CartItem`, `Order`/`OrderItem`/`OrderTracking`, `Payment`, `Coupon`, `Review`, `WishlistItem`.

Relationships follow the natural e-commerce shape: a `User` (customer) has `Addresses`, a `Cart` of `CartItems` (each pointing to a `ProductVariant`), and `Orders` composed of `OrderItems`. A `Product` belongs to a `Category`, has one or more `ProductVariants` (for size/color/SKU-level pricing and stock) and `ProductImages`. Sellers are `Users` with `Role.ROLE_SELLER` and an associated `SellerDetails` profile.

Known Limitations

- **Hardcoded secrets in `application.properties`:** default values are provided for `DB_PASSWORD` and `JWT_SECRET`. These must be overridden via environment variables in any non-local environment.

- **No CORS configuration observed** beyond the dev-time Vite proxy — deploying frontend and backend on different origins in production will require explicit CORS setup on the backend.
- **File uploads are stored on local disk** (`uploads/` directory) rather than object storage — this won't survive container restarts or scale across multiple backend instances without a shared volume.
- **No visible rate limiting** on auth endpoints (`/api/auth/signin`, `/signup`) — consider adding this before exposing the API publicly.
- **No API versioning** (`/api/...` rather than `/api/v1/...`) — breaking changes to DTOs will affect all clients immediately.

Troubleshooting / FAQ

"Connection refused" from the frontend calling the API Confirm the backend is running on port 8081 and `VITE_API_BASE_URL` (or the Vite proxy) points at the right host/port.

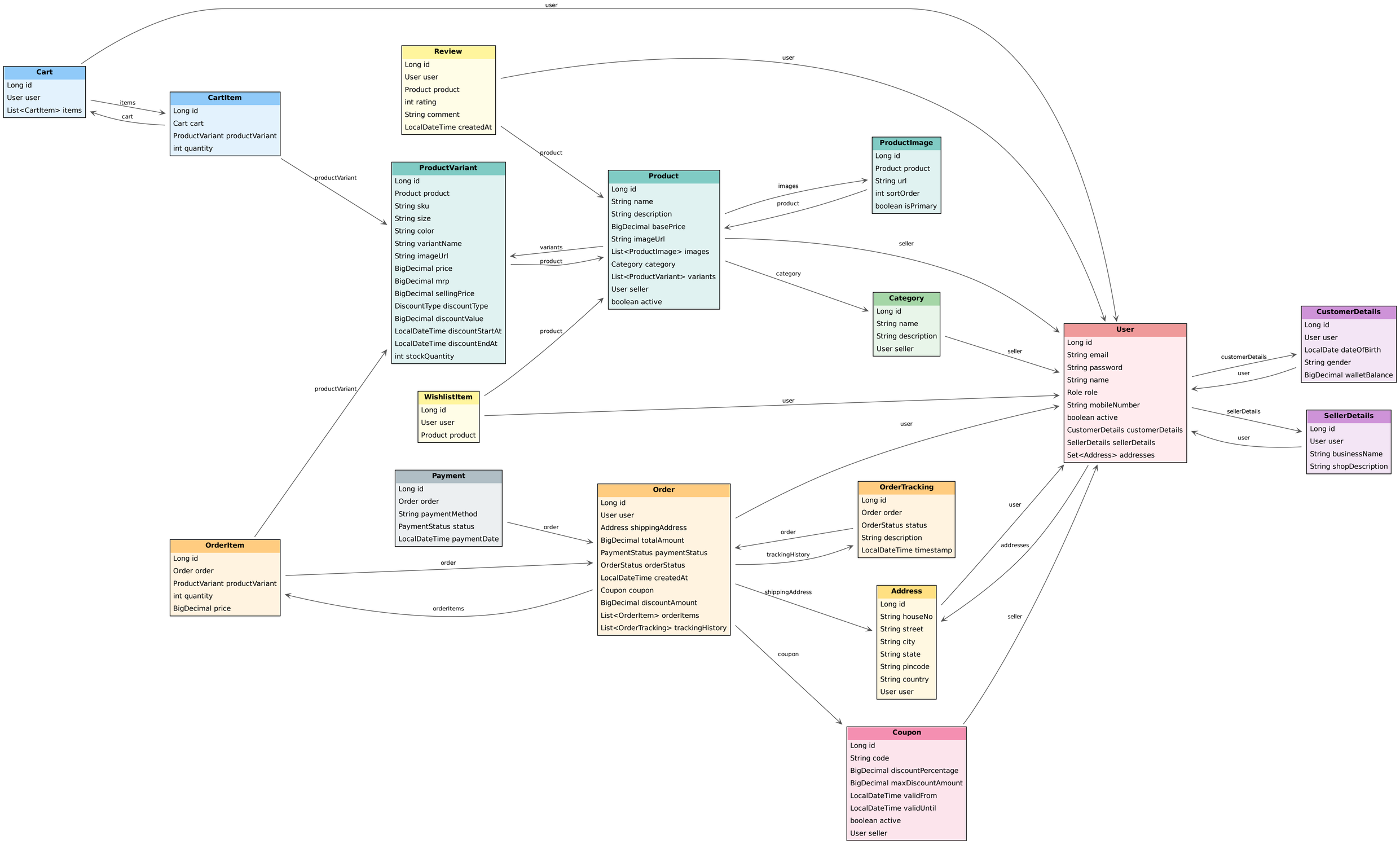
401 Unauthorized on requests that should be public Check `SecurityConfig` — endpoints not explicitly permitted may default to requiring authentication depending on the base security filter chain configuration.

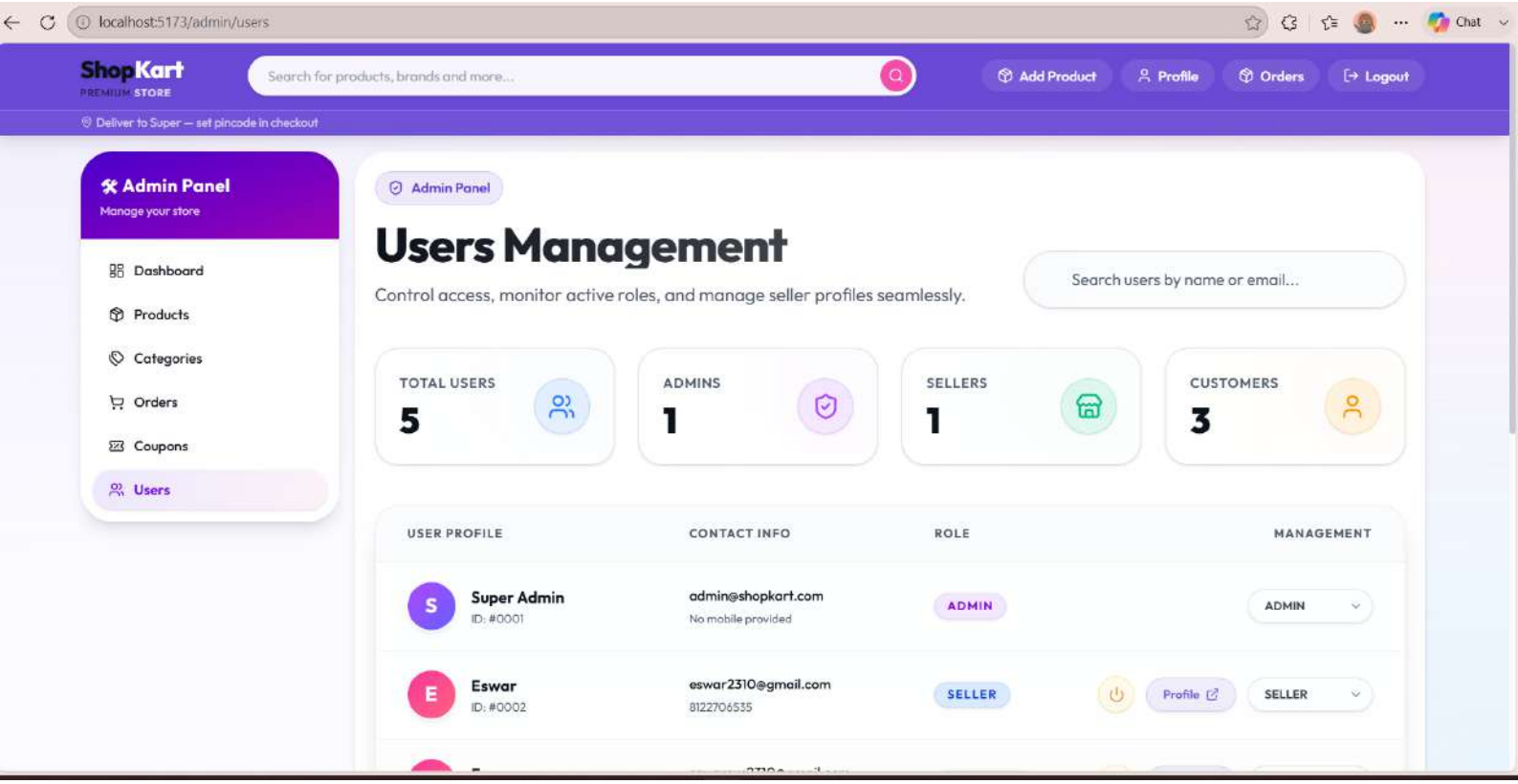
403 Forbidden on an endpoint you expect to access Check the `@PreAuthorize` annotation on that controller method/class against your account's role(s) in the JWT (`roles` field returned at signin).

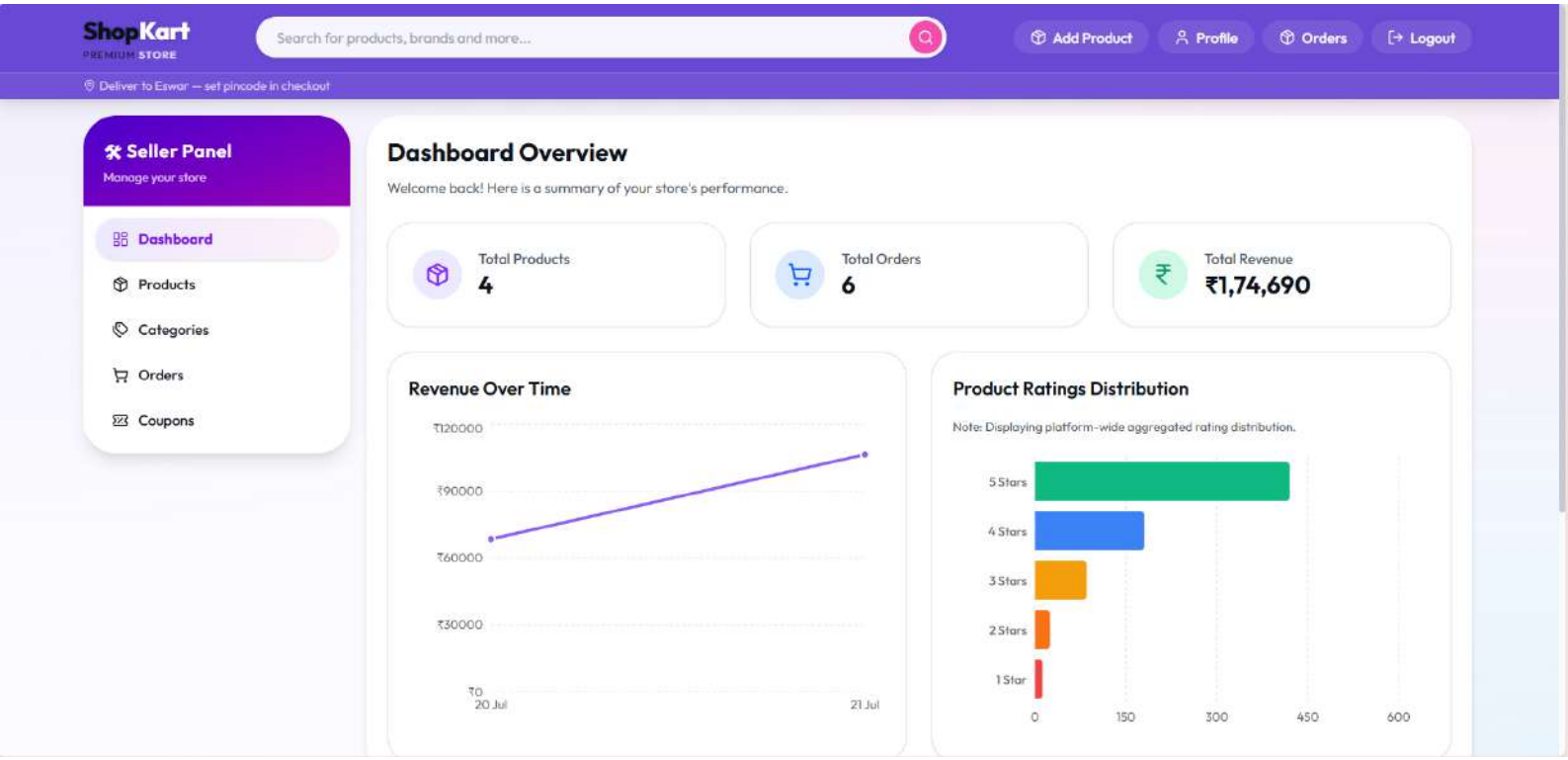
MySQL connection errors on startup Verify `DB_URL`, `DB_USERNAME`, and `DB_PASSWORD` match a running MySQL instance, and that the `e_commerce` database exists (Hibernate creates tables, not the database itself).

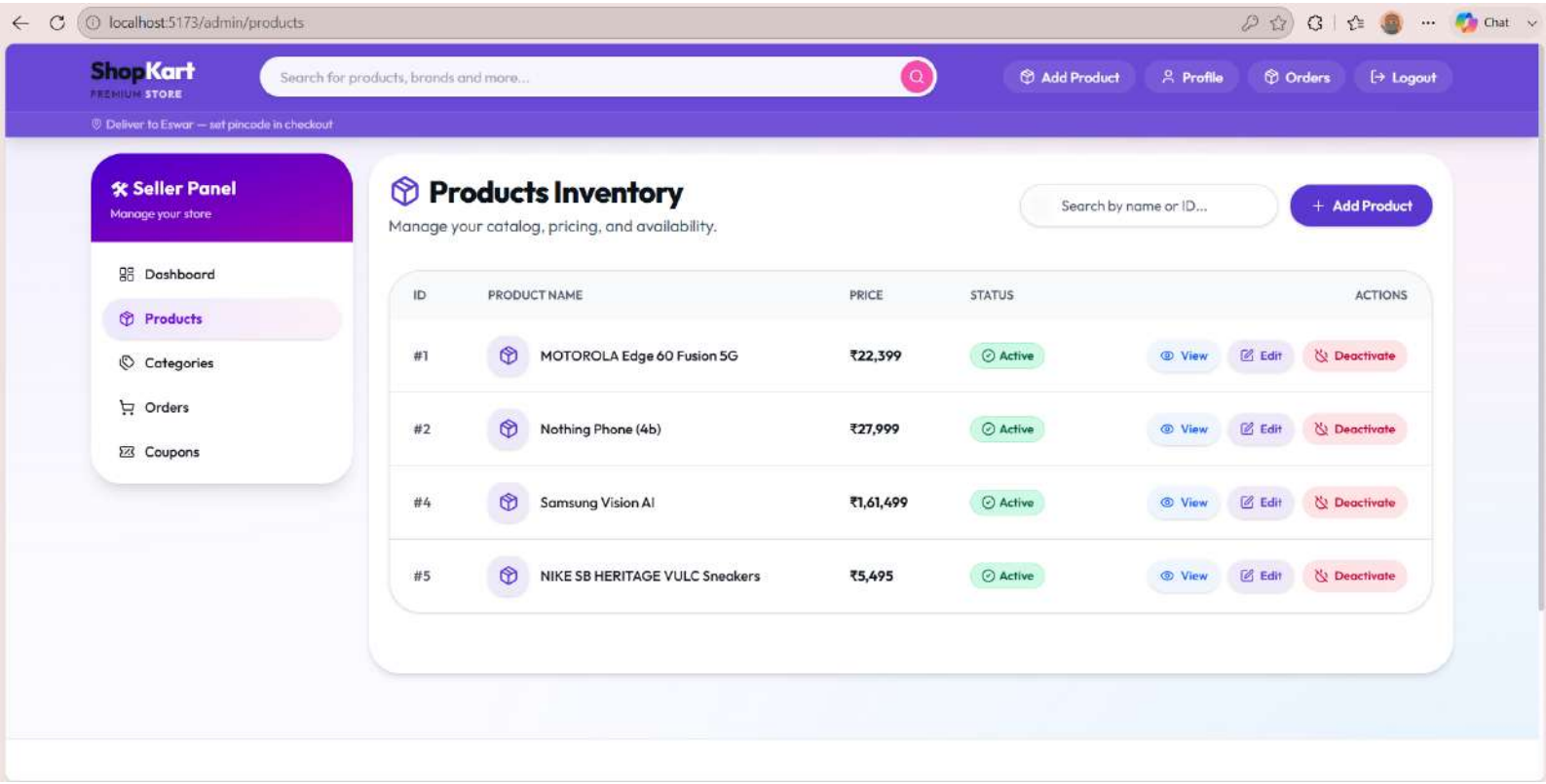
Uploaded images return 404 Check `WebMvcConfig` for how the `uploads/` directory is exposed as a static resource path, and confirm `UPLOAD_DIR` matches where files are actually being written.

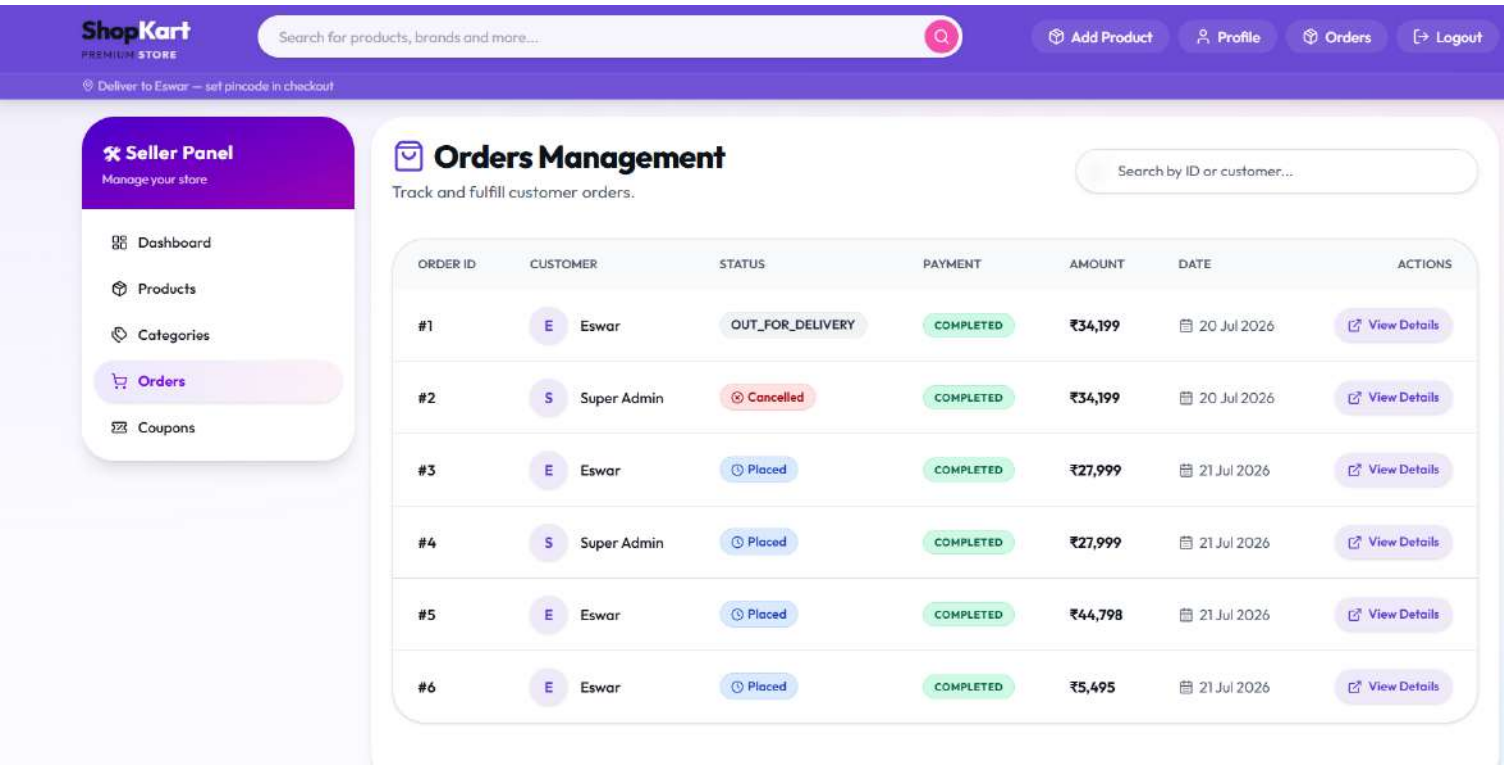
Password validation keeps failing on signup The regex requires 8+ characters with at least one uppercase letter, one lowercase letter, one digit, and one of `@#$%^&+=!`.


Cart
Long id
User user
List<CartItem> items
CartItem
Long id
Cart cart
ProductVariant productVariant
int quantity
Review
Long id
User user
Product product
int rating
String comment
LocalDateTime createdAt
ProductVariant
Long id
Product product
String sku
String size
String color
String variantName
String imageUrl
BigDecimal price
BigDecimal mrp
BigDecimal sellingPrice
DiscountType discountType
BigDecimal discountValue
LocalDateTime discountStartAt
LocalDateTime discountEndAt
int stockQuantity
Product
Long id
String name
String description
BigDecimal basePrice
String imageUrl
List<ProductImage> images
Category category
List<ProductVariant> variants
User seller
boolean active
ProductImage
Long id
Product product
String url
int sortOrder
boolean isPrimary
Category
Long id
String name
String description
User seller
User
Long id
String email
String password
String name
Role role
String mobileNumber
boolean active
CustomerDetails customerDetails
SellerDetails sellerDetails
Set<Address> addresses
CustomerDetails
Long id
User user
LocalDate dateOfBirth
String gender
BigDecimal walletBalance
SellerDetails
Long id
User user
String businessName
String shopDescription
WishlistItem
Long id
User user
Product product
Payment
Long id
Order order
String paymentMethod
PaymentStatus status
LocalDateTime paymentDate
Order
Long id
User user
Address shippingAddress
BigDecimal totalAmount
PaymentStatus paymentStatus
OrderStatus orderStatus
LocalDateTime createdAt
Coupon coupon
BigDecimal discountAmount
List<OrderItem> orderItems
List<OrderTracking> trackingHistory
OrderTracking
Long id
Order order
OrderStatus status
String description
LocalDateTime timestamp
OrderItem
Long id
Order order
ProductVariant productVariant
int quantity
BigDecimal price
Address
Long id
String houseNo
String street
String city
String state
String pincode
String country
User user
Coupon
Long id
String code
BigDecimal discountPercentage
BigDecimal maxDiscountAmount
LocalDateTime validFrom
LocalDateTime validUntil
boolean active
User seller
user
items
cart
user
product
productVariant
images
product
variants
product
seller
category
productVariant
product
user
seller
customerDetails
user
sellerDetails
user
user
order
order
orderItems
order
trackingHistory
shippingAddress
coupon
user
addresses
seller


localhost:5173/admin/users
Chat
ShopKart
PREMIUM STORE
Search for products, brands and more...
Add Product
Profile
Orders
Logout
Deliver to Super — set pincode in checkout
Admin Panel
Manage your store
Dashboard
Products
Categories
Orders
Coupons
Users
Admin Panel
Users Management
Control access, monitor active roles, and manage seller profiles seamlessly.
Search users by name or email...
TOTAL USERS
5
ADMINS
1
SELLERS
1
CUSTOMERS
3
USER PROFILE
CONTACT INFO
ROLE
MANAGEMENT
S
Super Admin
ID: #0001
admin@shopkart.com
No mobile provided
ADMIN
ADMIN
E
Eswar
ID: #0002
eswar2310@gmail.com
8122706535
SELLER
Profile
SELLER


ShopKart
PREMIUM STORE
Search for products, brands and more...
Add Product
Profile
Orders
Logout
Deliver to Eswar — set pincode in checkout
Seller Panel
Manage your store
Dashboard
Products
Categories
Orders
Coupons
Dashboard Overview
Welcome back! Here is a summary of your store's performance.
Total Products
4
Total Orders
6
Total Revenue
₹1,74,690
Revenue Over Time
₹120000
₹90000
₹60000
₹30000
₹0
20 Jul
21 Jul
Product Ratings Distribution
Note: Displaying platform-wide aggregated rating distribution.
5 Stars
4 Stars
3 Stars
2 Stars
1 Star
0
150
300
450
600


localhost:5173/admin/products
Chat
ShopKart
PREMIUM STORE
Search for products, brands and more...
Add Product
Profile
Orders
Logout
Deliver to Eswar — set pincode in checkout
Seller Panel
Manage your store
Dashboard
Products
Categories
Orders
Coupons
Products Inventory
Manage your catalog, pricing, and availability.
Search by name or ID...
+ Add Product
ID
PRODUCT NAME
PRICE
STATUS
ACTIONS
#1
MOTOROLA Edge 60 Fusion 5G
₹22,399
Active
View
Edit
Deactivate
#2
Nothing Phone (4b)
₹27,999
Active
View
Edit
Deactivate
#4
Samsung Vision AI
₹1,61,499
Active
View
Edit
Deactivate
#5
NIKE SB HERITAGE VULC Sneakers
₹5,495
Active
View
Edit
Deactivate


ShopKart
PREMIUM STORE
Search for products, brands and more...
Add Product
Profile
Orders
Logout
Deliver to Eswar — set pincode in checkout
Seller Panel
Manage your store
Dashboard
Products
Categories
Orders
Coupons
Orders Management
Track and fulfill customer orders.
Search by ID or customer...
ORDER ID
CUSTOMER
STATUS
PAYMENT
AMOUNT
DATE
ACTIONS
#1
E
Eswar
OUT_FOR_DELIVERY
COMPLETED
₹34,199
20 Jul 2026
View Details
#2
S
Super Admin
Cancelled
COMPLETED
₹34,199
20 Jul 2026
View Details
#3
E
Eswar
Placed
COMPLETED
₹27,999
21 Jul 2026
View Details
#4
S
Super Admin
Placed
COMPLETED
₹27,999
21 Jul 2026
View Details
#5
E
Eswar
Placed
COMPLETED
₹44,798
21 Jul 2026
View Details
#6
E
Eswar
Placed
COMPLETED
₹5,495
21 Jul 2026
View Details

ShopKart
PREMIUM STORE

Search for products, brands and more...

Add Product | Profile | Orders | Logout

Deliver to Eswar — set pincode in checkout

Trending Products

VIEW ALL >



4.2

MOTOROLA Edge 60 Fusion 5G

₹22,399 ~~₹37,999~~

41% OFF



4.2

Nothing Phone (4b)

₹27,999 ~~₹34,999~~

20% OFF



4.2

I phone 17 pro max

₹1,47,000



4.2

Samsung Vision AI

₹1,61,499 ~~₹1,89,999~~

15% OFF



4.2

NIKE SB HERITAGE VULC Sneakers

₹5,495 ~~₹8,999~~

39% OFF